

Adaptive iterative retrieval for enhanced retrieval-augmented generation

Wenhan Han^{a,*}, Xiao Xiao^b, Yaohang Li^c, Jun Wang^d, Mykola Pechenizkiy^a, Meng Fang^{b, a}

^a Eindhoven University of Technology, Groene Loper 3, Eindhoven, 5612AE, Netherlands

^b University of Liverpool, Foundation Building, Brownlow Hill, Liverpool, L69 7ZX, United Kingdom

^c University of Technology Sydney, 61 Broadway, Ultimo, 2007, NSW, Australia

^d University College London, Gower Street, London, WC1E 6BT, United Kingdom

HIGHLIGHTS

- We propose AIR-RAG, an adaptive, iterative framework for RAG optimization.
- AIR-RAG improves both retrieval relevance and LLM alignment without retraining.
- Our method uses adaptive feedback across iterations for better document selection.
- AIR-RAG integrates seamlessly with existing RAG pipelines.
- Experiments show that AIR-RAG outperforms strong baselines on multiple benchmarks.

ARTICLE INFO

Communicated by S. Mirjalili

Keywords:

LLM

RAG

ABSTRACT

Existing retrieval-augmented generation (RAG) methods often treat retrieval as a one-off operation, yet recent work suggests that iteratively refining the retrieval step can yield substantial gains in relevance and downstream generation quality. However, prior iterative-retrieval approaches typically optimize only the retriever's ranking function or only post-hoc document refinement, and they require expensive retriever retraining or complex multi-stage pipelines. To address these challenges, we propose Adaptive Iterative Retrieval for Retrieval-Augmented Generation (AIR-RAG), an adaptive, iterative retrieval framework designed to optimize both document relevance and LLM alignment within the RAG pipeline. By leveraging adaptive feedback, AIR-RAG simultaneously enhances retrieval ranking and document refinement across multiple iterations, eliminating the need for complex retraining pipelines and enabling seamless integration with existing systems. In extensive evaluations against state-of-the-art RAG methods across several benchmark datasets including TriviaQA, PopQA, HotpotQA, WikiMultiHop, PubHealth, and StrategyQA, AIR-RAG consistently demonstrates superior performance, underscoring its effectiveness in enhancing retrieval-augmented generation systems. Our code and data are available anonymously at <https://github.com/aialt/AIR-RAG>.

1. Introduction

Large language models (LLMs) often face challenges with outdated knowledge, and updating them through retraining is both costly and inefficient. Retrieval-Augmented Generation (RAG) offers a solution by retrieving relevant passages for a given query, enabling LLMs to incorporate up-to-date information and produce more accurate responses. RAG has demonstrated significant success across various natural language processing (NLP) tasks [1–5].

Iterative retrieval has emerged as an effective strategy to refine RAG systems. This approach involves constructing search paths or enabling

iterative interactions between the retriever and the LLM in an adaptive manner. However, many existing methods tend to overlook the alignment between the retriever and the LLM during each iteration of the RAG process. The lack of alignment can result in ineffective collaboration, diminishing both retrieval quality and answer relevance [6–8]. For example, Self-RAG [6] uses special tokens to let the LLM express its retrieval needs and assess document relevance. ActiveRAG [9] generates pseudo-sentences to analyze low-frequency tokens and refine retrievals for factual accuracy. Iter-RetGen [10] and Inter [11] iteratively refine queries with LLM-generated content, performing further retrievals until a final answer is reached. Methods like Self-Ask [8] and GenGround

* Corresponding author.

Email address: w.han@tue.nl (W. Han).

Table 1
Comparison of key RAG optimization approaches.

Method	Objective	Methodology
<i>Iterative RAG</i>		
Self-RAG [6]	Improve factuality and self-consistency	• Use special “reflection” tokens to decide when to retrieve. • Critique its own generated outputs.
ActiveRAG [9]	Reduce errors via anticipatory retrieval	• Predict low-confidence tokens during generation. • Generate pseudo-sentences as queries for targeted retrieval.
Iter-RetGen [10]	Boost relevance via gen-ret iteration	• Alternate between generation and retrieval rounds. • Use generated drafts to refine subsequent queries.
InteR [11]	Iteratively refine queries	• Use LLM-generated content to refine queries iteratively until the answer stabilizes.
Self-Ask [8]	Improve compositional reasoning	• Decompose complex questions into sub-questions. • Retrieve/answer each sub-part, then aggregate.
GenGround [12]	Strengthen grounding	• “Generate-then-ground”: Draft an answer first. • Retrieve facts to verify and refine the draft.
<i>Retriever optimization & query rewriting</i>		
RRR [13]	Align retriever with generator needs	• Rewrite queries using LLM feedback. • Retrieve with the new query, then generate the answer.
CONQRR [14]	Conversational query rewriting	• Use reinforcement learning to rewrite conversational history into stand-alone queries.
LLM4CS [15]	Capture dialog search intent	• Prompt an LLM to propose and merge multiple query rewrites to better represent user intent.
ARL2 [16]	Align retriever to LLM preferences	• Use LLM-labeled relevance data to fine-tune the retriever.
Search-Adaptor [17]	Customize embeddings for IR	• Adapt representations for the specific retrieval task (embedding customization).
D2LLMs [18]	Efficiency + nuanced relevance	• Distill signals from a powerful cross-encoder into a scalable bi-encoder retriever.
Landmark Embedding [19]	Chunk-free long-context retrieval	• Train window-aware embeddings to represent long contexts without chunking.
Multi-Task Embedder [20]	Generalize retriever signals	• Train text embedders with rank-aware rewards that incorporate LLM feedback.
RetrievalQA [21]	Decide when to retrieve	• Assess whether to use retrieval or the model’s parametric knowledge for short-form QA.
<i>Refinement (post-retrieval)</i>		
CFIC [22]	Sentence-level signal without chunks	• Perform chunking-free in-context retrieval. • Select sentences via constrained decoding or hidden states.
Info-RAG [23]	Robustness to noisy retrieval	• Use unsupervised information-refinement training to better fuse retrieved text with model knowledge.
BGM [7]	Close pref. gap in re-ranking	• A bridge model reorders/filters retrieved documents to suit LLM or task preferences.
Noise Filtering (IB)	Filter retrieval noise	• Employ an information-bottleneck based selection to retain only answer-salient content.
<i>Adaptive iterative retrieval alignment</i>		
AIR-RAG (ours)	Jointly align retriever and LLM	• Lightweight adapter selects sentences and augments queries across 2 iterations. • Trained with weighted Kahneman–Tversky Optimization (KTO) from LLM/retriever preferences. • No retraining of the base retriever or LLM is required.

[12] break down complex questions into sub-questions, repeating the RAG process to generate the answer. Ke et al. [7] only considered the preference of the LLM but overlooked that in the retriever. Overall, these methods fail to address the critical alignment of the preferences between both the retriever and the LLM.

In this paper, we propose Adaptive Iterative Retrieval for Retrieval-Augmented Generation (AIR-RAG), an adaptive, iterative retrieval framework that optimizes both the relevance of retrieved documents and the alignment with LLMs within the RAG pipeline. Rather than focusing solely on improving retriever performance or post-retrieval document refinement, our approach adaptively adjusts the retrieval process, aligning it with the needs of both the retriever and the LLM. This iterative refinement ensures that the retrieved context is progressively enhanced, maximizing its relevance to both the query and the LLM’s task.

In our approach, the retriever initially fetches a set of candidate documents based on the original query. The LLM processes these documents and provides feedback, which informs the next iteration of retrieval, ensuring that more relevant documents are selected. This process is repeated iteratively, with each round of retrieval improving the context provided to the LLM, ultimately enhancing the quality and accuracy

of the generated output. Our method eliminates the complexity of re-training both the retriever and LLM simultaneously, offering a more efficient and scalable solution. Table 1 presents the comparison between AIR-RAG and previous RAG optimization approaches.

Summary of Contributions:

- We introduce AIR-RAG, an adaptive, iterative retrieval framework that enhances both retrieval relevance and LLM alignment, optimizing the RAG pipeline.
- Our method leverages adaptive feedback across multiple iterations to improve retrieval ranking and document refinement, enabling seamless integration with existing systems.
- We conduct extensive experiments and demonstrate that AIR-RAG consistently outperforms existing RAG methods, highlighting its superior performance across several benchmark datasets.

2. Related work

Iterative RAG. This approach often involves constructing search paths or iteratively interacting between the retriever and the LLM in an adaptive manner. Self-RAG [6] introduces special tokens that enable the LLM

to automatically express its retrieval needs and assess the relevance of retrieved documents. ActiveRAG [9], on the other hand, prompts the LLM to generate a pseudo-sentence, analyzes low-frequency tokens within it, and conducts targeted retrievals to correct factual inaccuracies. IterRetGen [10] and Inter [11] iteratively refine the query using content generated by the LLM, performing additional retrievals until a final answer is produced. Self-Ask [8] and GenGround [12] decompose complex questions into simpler sub-questions, repeating the RAG process until a final answer is reached. However, these methods often require numerous iterations and tend to overlook alignment between the retriever and LLM during each RAG operation. In contrast, our approach focuses on optimizing every minimal unit within the RAG pipeline, ensuring better coordination and performance throughout the entire process. BGM [7] identifies a preference gap between retrievers and LLMs, and proposes refining retrieved documents by aligning them with downstream task preferences. Their method focuses on document-level re-ranking after retrieval, whereas our approach introduces an intermediate adapter model that iteratively aligns both retriever and LLM preferences at a finer granularity. In parallel, RetrievalQA [21] shifts the focus from optimizing retrieval quality to deciding whether retrieval is needed at all.

Retriever optimization. D2LLMs [18] combine the efficiency of bi-encoders with the nuanced understanding of LLMs in semantic search by decomposing and distilling an LLM cross-encoder into a bi-encoder. Search-Adaptor [17] customizes the embeddings generated by LLMs for retrieval tasks. Landmark Embedding [19] introduces a three-stage method to train LLaMA-2 [24], enabling it to embed sentences within a window of context for retrieval purposes. ARL2 [16] aligns retrievers with LLMs by leveraging LLM-labeled relevance training data. Additionally, Zhang et al. [20] train a multi-task embedder using a rank-aware reward that incorporates LLM feedback. However, all of these methods require retraining the retriever, which is computationally expensive. In contrast, our approach can utilize any off-the-shelf retriever, eliminating the need for retriever retraining and significantly reducing computational costs.

Beyond retriever training, some studies focus on rewriting queries to enhance retrieval quality. For instance, CONQRR [14] and RRR [13] apply reinforcement learning to optimize query rewriting models based on feedback. Without requiring additional training, LLM4CS [15] prompts the LLM to generate multiple query rewritings and synthesizes their embeddings as input for the retriever. While query rewriting can help address the alignment issue between the retriever and LLM, it falls short in performing fine-grained filtering of retrieval results, leaving noise in the retrieved documents unaddressed.

Refinement. Refinement involves extracting useful information from lengthy retrieved documents. Zhu et al. [25] propose filtering noise by minimizing the mutual information between the refined text and the retrieved passages, while maximizing the mutual information between the refined text and the true answer. Similarly, CFIC [22] trains a refiner by selecting sentences based on the generation probabilities of prefix tokens. BGM [7] selects documents from the retrieved set and aligns them with downstream task metrics. Beyond refinement, Info-RAG [23] integrates the knowledge from the retrieved passages with the LLM's parameters to improve its robustness when dealing with noisy retrieval results. Fang et al. [26] also aim to enhance LLM robustness by constructing a noisy dataset and using adversarial training. However, these methods focus exclusively on making the LLM a better document reader, while overlooking the retrieval preferences of the upstream retriever.

3. Preliminary

Preference training. Aligning a model π_θ with the preferences is to learn the value function $v(r_\theta(q, y) - \mathbb{E}_Q[r_\theta(q, y')])$, where $r_\theta = \log \frac{\pi_\theta(y|q)}{\pi_{\text{ref}}(y|q)}$ is an implicit reward function, q is the query, y is the response and π_{ref} is the

reference model [27]. $\mathbb{E}_Q$ represents the reference point and $Q(Y|q)$ is a reference point distribution. The loss then is defined as

$$\mathcal{L}(\pi_\theta, \pi_{\text{ref}}) = \mathbb{E}_{q, y \sim D} [a_{q, y} v(r_\theta(q, y) - \mathbb{E}_Q[r_\theta(q, y')])], \quad (1)$$

where $a_{q, y} \in \{-1, +1\}$ represents the preferences and y' is the other possible generations. Inspired by this approach, we use it as the LLM-aware (or retriever-aware) loss to align the components in the RAG pipeline.

4. Method

4.1. Overview

Our approach improves the synergy between the retriever and the LLM generator used for answer generation through adaptive iterative retrieval. We propose a method that involves an intermediate model to control the data flow in the retrieval pipeline. This model helps the retriever accurately find useful passages and provides context adapted for the LLM, enabling the LLM to generate more accurate answers. The process starts by collecting preference data from both the LLM and the retriever, followed by training the model offline. Once trained, the model is integrated into the RAG pipeline, working collaboratively with the retriever and the LLM to optimize the retrieval process.

Fig. 1 illustrates how our method works. Given a query q , the retriever R first returns a set of top- k passages. Each passage is assigned a unique sentence ID (SID), and these top- k passages are rewritten as $Top-k = \{SID_1 \oplus S_1, SID_2 \oplus S_2, \dots, SID_n \oplus S_n\}$, where $\oplus$ denotes concatenation. The intermediate model processes these passages to select the most relevant sentence IDs: $M(q, Top-k) = \{SID_1, SID_2, \dots, SID_m\}$. This selected set is used to augment the original query: $q' = q \oplus M(q, Top-k)$, triggering the second iteration of retrieval and returning a new set of passages $Top-k' = R(q')$. These passages undergo another filtering step using the intermediate model, and the selected sentences are passed to the LLM for final answer generation: $Answer = LLM(q, M(q', Top-k'))$. Note that the sentence IDs are replaced with the actual sentences during processing. The detailed process of AIR-RAG is elaborated in Algorithm 1.

4.2. Preference collection

In the RAG pipeline, learning the LLM's preference involves understanding how to craft prompts that guide the LLM to generate the desired answer. Learning the retriever's preference focuses on determining how to refine queries so that the retriever produces high-quality retrieval results. The desired answer is well-defined because it can be easily compared against the ground truth. However, the retriever's output lacks

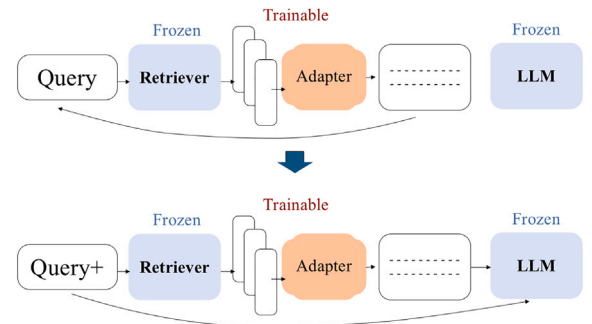


Fig. 1. Our pipeline involves two iterations. In the first iteration, the intermediate model filters the initial retrieval results, selecting context that is beneficial for answering the query and improving the retrieval results. In the second iteration, the enhanced query is used to perform a second retrieval, followed by another round of filtering, and then the LLM generates the final answer. The orange line represents the collection of preference data from the LLM and retriever, which is used to train the model.

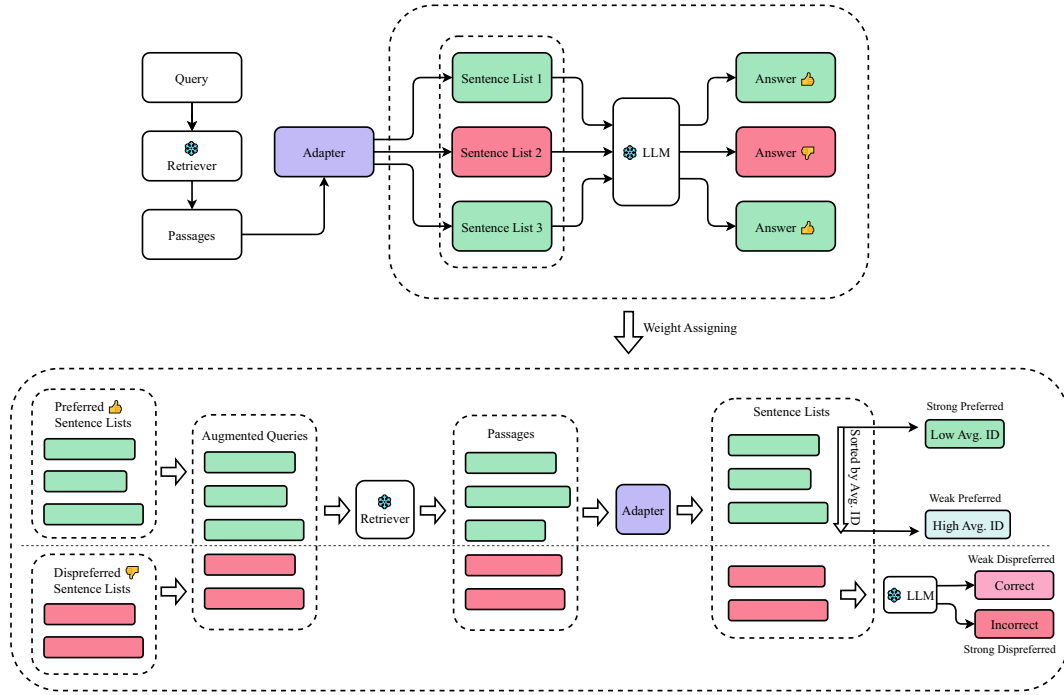


Fig. 2. Preference collection. The collection process is divided into two steps, corresponding to two iterations of inference. In the upper part of the diagram, the selection of the adapter is labeled as either “preferred” or “dispreferred.” In the lower part, weights are assigned to the samples of these two labeled groups.

explicit labels, and relevant document labels are scarce and often manually annotated, which can be costly. Using LLMs to directly annotate documents for relevance is computationally expensive and may introduce bias, as it does not align with the final task objectives. Therefore, we propose an approach to implicitly collect retriever preferences and align them directly with the LLM’s output objectives. Fig. 2 depicts the data collection procedure. In this setting, we annotate the samples with 4 labels, *strongly preferred*, *weakly preferred*, *weakly dispreferred*, and *strongly dispreferred*.

LLM preference. Given a query q and selected relevant sentences $y = \{s_1, s_2, \dots, s_m\}$, the selection is labeled as preferred if the LLM answers the query correctly, otherwise it is labeled as dispreferred. Once an LLM preference dataset $D_l = \{(q_i, y_i, a_i)\}_{i=0}^n$, where $a_i \in \{-1, +1\}$, is collected, the intermediate model learns a policy π_θ to minimize the loss as defined in Eq. (1).

Retriever preference. Similarly, a sentence selection is considered preferred if the augmented query results in better retrieval outcomes in the second iteration. However, golden passages are not always easily obtainable and are often manually annotated, which incurs significant workload and costs. Therefore, we indirectly collect the retriever’s preferences by comparing the answers generated by the LLM using the results from the two retrieval iterations and analyzing the distribution of useful information within the retrieved documents. The retriever preference dataset is similarly denoted as $D_r = \{(q_i, y_i, a_i)\}_{i=0}^n, a_i \in \{-1, +1\}$.

Directly synthesizing the two datasets D_l and D_r for training the model would introduce noise into the preferences of both the LLM and the retriever. To mitigate this, we extend the preference labels to a four-value system: *strongly preferred*, *weakly preferred*, *weakly dispreferred*, *strongly dispreferred*. If both the LLM and retriever prefer the sentence selection, it is labeled as strongly preferred. If only the LLM prefers it, it is weakly preferred. If only the retriever prefers it, it is weakly dispreferred. If neither prefers it, it is strongly dispreferred. This extended label system helps to combine D_l and D_r into a unified dataset D , integrating the preferences of both the LLM and the retriever.

We describe the workflow of preference data collection. Given a query q , we first sample multiple sentence lists $Y = \{y_i | y_i \sim M(q, Top-k)\}_{i=0}^n$ from the intermediate model. The LLM generates answers based on these lists, and they are initially labeled as preferred or dispreferred. Next, we assign weights (strong or weak) to these labeled lists in a second iteration. These lists are added to the original query, forming augmented queries $\{q'_i\}_{i=0}^n$. The retriever uses these augmented queries to retrieve relevant passages, and the intermediate model generates new sentence lists $Y' = \{y'_i\}_{i=0}^n$. For each augmented query, only one sentence list is generated. The newly generated lists are divided into two groups (preferred and dispreferred) based on the initial label, and then the preference weights are assigned according to the final output of the LLM given the sentence lists.

4.3. Model optimization

The intermediate model is tasked with two optimization objectives: aligning knowledge with the preferences of the LLM and offering query background aligned with the preferences of the retriever. We focus on achieving these two objectives within a single model, simplifying the overall pipeline and reducing the difficulty of optimization. Sentence IDs serve as an information bottleneck, limiting the action space of the model, which helps control the behavior of both the LLM and the retriever, simplifying its optimization from preference data.

Weighted Kahneman-Tversky optimization. Aligning large language models with human preferences has traditionally relied on complex methods like Reinforcement Learning from Human Feedback (RLHF) [28] or approaches such as Direct Preference Optimization (DPO) [29], which require costly paired-preference annotations. Kahneman-Tversky Optimization (KTO) offers a more data-efficient alternative [27]. The key innovation of KTO is its ability to learn from simpler binary feedback indicating whether a given generation is “desirable” or “undesirable”, avoiding the need for direct pairwise comparisons. This approach is grounded in the principles of prospect theory, which models human decision-making biases such as loss aversion. By directly maximizing the utility of generations instead of the log-likelihood of preferences, KTO

provides a robust and efficient method for model alignment, making it particularly well-suited for our framework. We therefore adopt KTO to optimize the model, as it simplifies the framework's complexity and improves data efficiency. The KTO method estimates the expectation $\mathbb{E}_Q$ of the reward function r_θ in a biased manner, defined as:

$$\hat{z}_0 = \max \left(0, \frac{1}{m} \sum_{i \neq j} \log \frac{\pi_\theta(y_j | q_i)}{\pi_{\text{ref}}(y_j | q_i)} \right),$$

where q_i and y_j are mismatched within a batch of samples. The biased estimate is less significant due to the limited action space of the model, making the estimation of $\mathbb{E}_Q$ more accurate. We also extend KTO to handle the four-value preference system, adjusting the training loss for the model:

$$\mathcal{L}_M(\pi_\theta, \pi_{\text{ref}}) = \mathbb{E}_{q, y \sim D} [w \cdot a_{q, y} v(r_\theta(q, y) - \hat{z}_0)], \quad (2)$$

where $a_{q, y} \in \{-1, +1\}$ is the preference label, and $w \in \{0.5(\text{weak}), 1(\text{strong})\}$.

5. Experiments

We design experiments to compare the performance of our proposed method, Adaptive Iterative Retrieval, with several other RAG methods.

5.1. Setup

Datasets. We perform evaluations on six datasets across three tasks: (1) OpenQA: **TriviaQA** [30] and **PopQA** [31]; (2) Multi-hop QA: **HotpotQA** [32] and **WikiMultiHopQA** [33]; (3) Fact-checking: **PubHealth** [34] and **StrategyQA** [35]. We only used the training data from TriviaQA and HotpotQA to optimize the adapter model, while other test datasets were treated as independent evaluations. Due to the time-consuming testing process, we only used the first 1000 queries from each dataset.

Preference collection data. We use the training sets of TriviaQA [30] and HotpotQA [32] for preference data collection, leveraging only the queries and answers from these datasets. The *LLaMA-3.1-8B-Instruct* model is used as the intermediate model for generating sentence selections. To fine-tune the LLM for answer generation, we follow the approach in [36], using *LLaMA-3-8B* as the answer generator. As the LLM's output format can vary, we also use *ChatGPT* to assess answer correctness, mitigating issues of format mismatches. For each query, we generate 5 samples. If all the answers are correct, we discard the query, as it is deemed too simple to require optimization. Similarly, if all 5 samples are dispreferred and the answers remain incorrect after the second iteration, the query is discarded as too difficult for effective model training. The statistics of the collected preference data are presented in Table 2. The difficulty of each query is determined by the percentage of incorrect answers among the five sentence selection samples. During the data collection process, we observed that TriviaQA generated more samples with entirely correct answers (which were discarded) than HotpotQA, indicating that TriviaQA is easier. Therefore, to balance data collection efficiency and difficulty, we collected 5 k samples from TriviaQA and 12 k samples from HotpotQA.

Metrics. We use Accuracy (ACC) as the metric to evaluate the model's responses, which is determined by checking whether the correct answer is included in the model's generated content. However, due to the variability in the model's responses, the Accuracy metric may introduce errors. Similar to the strategy used when collecting preference data, we additionally use ChatGPT to assess whether the model's response aligns with the reference answer, thereby calculating the **Correctness** score.

5.2. Baselines

We compare Adaptive Iterative Retrieval with several baselines: (1) **Direct**: The LLM is prompted to answer the question without retrieval; (2) **NaiveRAG**: A standard RAG process, where the retrieved

Table 2

Preference data statistics.

	Query source		
	TriviaQA	HotpotQA	Overall
Samples	5006	12,805	17,811
Strong / Weak Pref.	1396 / 947	3466 / 2278	4862 / 3225
Strong / Weak Dispref.	1290 / 1373	3353 / 3708	4643 / 5081
Avg. Difficulty	52.19	54.61	53.93
Avg. Prompt Length	1173.67	1177.74	1176.59
Avg. Sentences	63.68	65.36	64.88
Avg. Selected Sentences	3.65	3.81	3.76

passages are passed to the LLM without any augmentation; (3) **Rewrite-Retrieve-Read (RRR)** [13]: A method that aligns the retriever and LLM through query rewriting; (4) **Iter-RetGen** [10]: A method that iteratively refines the query using content generated by the LLM; (5) **ActiveRAG** [9]: A method that predicts the next sentence to anticipate future content and uses it as a query to retrieve documents, regenerating the sentence if low-confidence tokens are detected; (6) **Self-Ask** [8]: A method that improves chain-of-thought reasoning by proposing follow-up sub-questions to solve before reaching a final answer; (7) **SelfRAG** [6]: Enhances RAG performance through adaptive retrieval and self-reflection; (8) **SearChain** [37]: Builds a reasoning chain to iteratively propose unsolved sub-questions and verify answers with retrieved information.

For SelfRAG, which requires additional special tokens for the LLM, we follow Zhang et al. [36] to fine-tune unified generators using the same training data as [6]. We also use ChatGPT as a generator to evaluate different methods, and reproduce SearChain with ChatGPT as the backend.

5.3. Implementation details

The adapter model training runs on a 4-GPU H100 node. The base model is *LLaMA-3.1-8B-Instruct*.¹ For a sufficient KL-divergence estimate in a KTO step, the batch size is set to 2 per GPU with 4 gradient accumulation steps. Low-Rank Adaptation (LoRA) is utilized where the rank and the scaling factor are 16, targeting all linear layers. The learning rate is $1e-5$, with a warmup ratio of 0.1. For preference data collection, *GPT-3.5-turbo* serves as the adapter model to generate sentence selections and as the discriminator to evaluate the correctness of the responses. ColBERTv2 [38] is employed as the retriever. We use the 2018 Wikipedia corpus provided by Karpukhin et al. [39], where the documents are chunked into passages with a maximum length of 100 words. For all methods we use the top-10 retrieved passages.

5.4. Results

Table 3 gives the results of the performance evaluation of AIR-RAG and the baselines. Our findings are as follows:

AIR-RAG is more effective than other RAG methods. It can be observed that our approach outperforms the previous RAG methods on most test sets and metrics. Apart from the TriviaQA and HotpotQA datasets, where the adapter model was trained using their training sets, significant improvements were also observed on other test sets. Moreover, AIR-RAG demonstrated high stability when handling different types of questions.

Strong direct and NaiveRAG are strong baselines. We observed that both Direct and NaiveRAG exhibited strong performance across multiple test sets. Nevertheless, AIR-RAG shows significant advantages over other methods. On PopQA (ACC), TriviaQA (ACC and Correctness), HotpotQA (Correctness) and WikiMultiHop (Correctness), it is the only method (with LLaMA-3 as the generator) that surpasses both Direct and

¹ <https://huggingface.co/meta-llama/Meta-Llama-3.1-8B-Instruct>.

Table 3

Performance comparison between AIR-RAG and various RAG methods. † indicates we do not use the training sets of those datasets to optimize the adapter model. The best scores are highlighted in bold, while the second-best scores are underlined. * denotes the result significantly outperforms the baselines by t -test ($p < 0.05$).

Method	PopQA†		TriviaQA		HotpotQA		WikiMultiHop†		PubHealth†	StrategyQA†
	ACC	Correctness	ACC	Correctness	ACC	Correctness	ACC	Correctness	ACC	ACC
<i>w/ LLaMA-3-8B</i>										
Direct	29.2	34.1	<u>66.7</u>	<u>61.6</u>	20.1	40.6	25.0	<u>37.3</u>	73.2	55.7
NaiveRag	<u>36.6</u>	43.7	<u>63.2</u>	<u>61.0</u>	26.7	<u>47.5</u>	20.1	32.0	67.8	57.8
RRR	35.9	43.0	58.8	56.0	23.2	42.8	21.9	29.8	68.3	58.2
Iter-RetGen	34.2	43.5	64.9	61.0	<u>27.0</u>	47.0	20.6	32.9	41.8	55.8
ActiveRAG	35.2	<u>44.6</u>	64.1	60.9	26.5	47.3	18.7	30.1	47.6	56.2
SelfAsk	11.2	16.6	36.0	36.1	14.6	30.2	17.6	26.9	40.6	52.1
SelfRAG	34.9	37.4	56.0	50.8	21.8	38.4	20.4	22.1	64.9	46.7
AIR-RAG (ours)	39.1*	45.7*	67.9*	65.5*	29.8*	54.8*	<u>22.7</u>	38.6*	<u>72.1</u>	<u>58.1</u>
<i>w/ GPT-3.5-turbo</i>										
Direct	32.0	37.0	77.5	71.7	31.8	54.4	37.0	42.7	77.7	68.0
NaiveRag	44.0	45.6	72.6	66.5	38.4	58.9	32.7	37.5	53.9	61.2
RRR	<u>44.7</u>	<u>46.2</u>	71.9	65.9	38.0	58.4	31.2	36.5	53.7	<u>63.3</u>
Iter-RetGen	43.7	45.1	73.5	67.5	43.7	<u>61.0</u>	35.8	38.5	42.6	56.9
ActiveRag	43.7	45.1	73.6	67.8	42.8	<u>61.0</u>	35.0	39.2	50.4	62.0
SelfAsk	35.9	41.4	66.3	62.0	36.7	57.2	39.1	42.8	46.3	38.4
SearChain	31.7	43.7	66.3	63.9	33.5	59.2	32.0	<u>44.5</u>	30.1	60.5
AIR-RAG (ours)	44.8	48.2*	<u>73.8</u>	<u>69.3</u>	<u>42.8</u>	62.8*	<u>37.3</u>	48.0*	<u>64.3</u>	61.9

NaiveRAG. This is because the LLM's parameters already contain knowledge relevant to these datasets. When provided with retrieved context, the LLM tends to suppress its parametric knowledge [40]. When other RAG methods supply context that isn't well-suited to the LLM, they can interfere with its performance, whereas AIR-RAG greatly reduces this interference. Another reason could be that the 2018 Wikipedia corpus lacks relevant documents. For instance, PubHealth involves a substantial amount of biomedical knowledge, which the retriever may struggle to provide effectively. In contrast to other RAG methods, AIR-RAG can adaptively filter out information that may interfere with the LLM's response.

AIR-RAG has strong generalization ability. Although the adapter model was trained based on the preferences of LLaMA-3-8B, it still demonstrates good generalization performance when the generator is replaced with GPT-3.5-turbo. This is because, during preference learning, the adapter model develops a stronger ability to filter out irrelevant information, which is a key factor in improving the performance of the RAG pipeline. As a result, an adapter model trained on the preferences of one LLM can still provide benefits to other LLMs.

6. Ablation study

In this section, we conduct a series of ablation studies, including comparisons between SFT and KTO optimization, the effect of weighted KTO training, scaling with respect to model size and data size, the impact of the number of iterations, and an analysis of computational latency.

6.1. Effectiveness of adapter training

We compare the impact of the adapter model trained with KTO and its base model (LLaMA-3.1) on the performance of AIR-RAG. We

also used GPT-3.5 as an adapter model for evaluation. Table 4 presents the results. It can be observed that preference alignment significantly improves the performance of LLaMA-3.1. Without training, LLaMA-3.1 underperforms NaiveRAG across all tasks. It can also be observed that using a stronger model (GPT-3.5-turbo) did not result in a substantial performance improvement. This highlights the importance of eliminating the preference gap between the retriever and the LLM through preference learning. Furthermore, this underscores that AIR-RAG is not merely a simple refinement or rewriting approach.

6.2. Effectiveness of weighted KTO

We retrained the adapter model with the exact same settings, but ignored the sample weights during loss calculation. Fig. 3 exhibits the difference between weighted and non-weighted KTO methods. The weighted KTO training demonstrates a clear advantage over the non-weighted version. By further distinguishing between weak and strong preferences in the binary preference data, the distraction of weak preference samples is reduced, allowing the adapter model to converge more effectively. Additionally, the non-weighted trained adapter model still shows improvements over NaiveRAG.

We also train an adapter model with SFT. Fig. 3 demonstrates a significant performance degradation with SFT. SFT provides an unbiased estimation of the target preference, capturing general trends in labeled data. However, it offers a biased estimation for the model, as it does not account for the model's inherent limitations or traits. This limitation means that SFT cannot enable the model to precisely adapt to subtle differences in preferences.

6.3. Model and training data size scaling

We additionally trained two adapter models based on Qwen-2.5: a 0.5B and a 1.5B model. These were compared with our adapter model

Table 4

Performance comparison between the adapter model and the base model.

Model	PopQA		TriviaQA		HotpotQA		WikiMultiHop		PubHealth	StrategyQA
	ACC	Correctness	ACC	Correctness	ACC	Correctness	ACC	Correctness	ACC	ACC
GPT-3.5-turbo	33.9	42.1	66.5	63.3	27.4	49.4	19.9	35.0	69.6	57.5
LLaMA-3.1-8B-Instruct	35.6	42.3	63.6	60.9	24.3	47.5	24.9	37.8	69.6	57.4
AIR-RAG	39.1	45.7	67.9	65.5	29.8	54.8	22.7	38.6	72.1	58.1

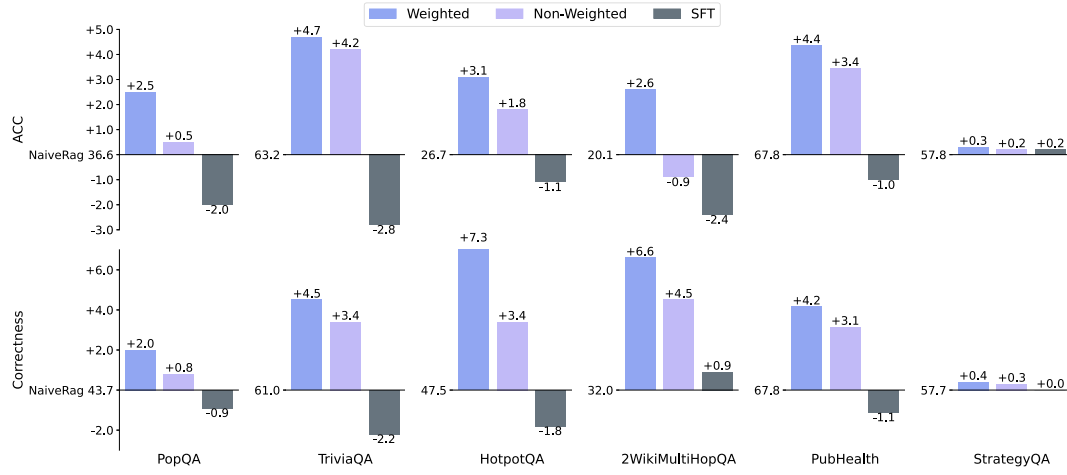
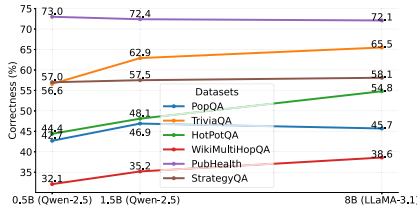
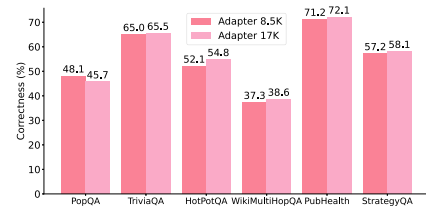


Fig. 3. Comparison between weighted KTO, non-weighted KTO and SFT trained AIR-RAG.



(a) Percentage change of different model sizes relative to 0.5B model



(b) Adapter models on different training dataset sizes.

Fig. 4. Correctness comparisons between different model and training data sizes.

Table 5
Iterations of AIR-RAG comparison.

Iterations	HotpotQA		WikiMultiHopQA		PopQA		TriviaQA	
	ACC	Correctness	ACC	Correctness	ACC	Correctness	ACC	Correctness
1	28.4	52.9	19.1	34.7	38.7	46.4	68.3	65.7
2	29.8	54.8	22.7	38.6	39.1	45.7	67.9	65.5
3	29.6	52.6	21.2	36.2	37.1	44.8	68.4	64.3

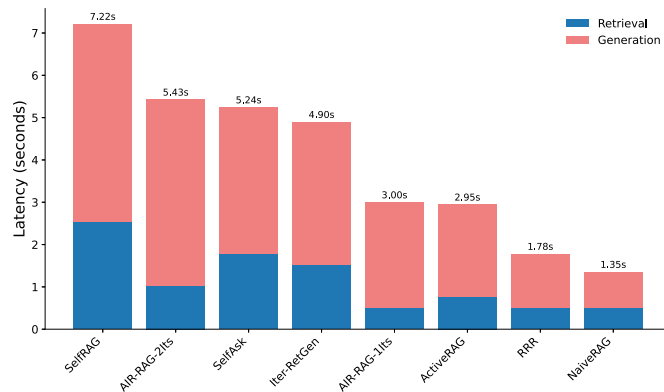


Fig. 5. The average processing latency of each sample.

based on LLaMA-3.1. From Fig. 4(a), it can be observed that as the model size increases, the performance also shows improvement. The 0.5B model has performance limitations, indicating that regulating the retriever and LLM within the RAG pipeline requires the adapter model to possess a considerable level of text understanding. Additionally, it is evident that the 1.5B model demonstrates quite strong performance,

slightly lagging behind the 8B model across datasets but still outperforming other RAG methods. This indicates that AIR-RAG is more efficient compared to other RAG methods. By using a smaller model as the adapter model to filter sentences, the token sequence length input to the LLM generator is significantly reduced. As a result, despite requiring two iterations, AIR-RAG has light resource demands.

To study the impact of training data sizes on the adapter model. We retrained the adapter model based on LLaMA-3.1-8B with half the preference data (8.5 K). The results are shown in Fig. 4(b). The performance gap between the adapter model trained with 8.5 K data and the one trained with 17 K data is minimal, demonstrating the high data efficiency of our proposed preference data collection method and the Weighted KTO training algorithm. AIR-RAG requires only a small amount of preference data to achieve a high level of performance.

6.4. Effectiveness of different iterations

Table 5 reveals the performance of AIR-RAG when different number of iterations are applied. Only one iteration means there is no query augmentation and a second retrieval, where the adapter model only functions as an information filter. It can be observed that the 2-Iteration AIR-RAG achieves the best overall performance. Information that was not accurately retrieved in the first iteration is often identified after an

Table 6

Two examples demonstrating retrieval-generation alignment.

Query	Which film was Oscar nominated, LaLee's Kin: the Legacy of Cotton or Gimme Shelter, the 1970 Rolling Stones documentary?
Iter-1 Sentences	s₂₄ Gimme Shelter is a 1970 British-American documentary film directed by Albert and David Maysles and Charlotte Zwerin, chronicling the last weeks of The Rolling Stones' 1969 US tour, which culminated in the disastrous Altamont Free Concert s₂₆ Gimme Shelter was screened out of competition as the opening film of the 1971 Cannes Film Festival
Iter-1 Answer	Gimme Shelter
Iter-2 Sentences	s ₈ LaLee's Kin: The Legacy of Cotton garnered a nomination for Best Documentary Feature at both the 74th Academy Awards and the 2002 Independent Spirit Award
Iter-2 Answer	LaLee's Kin: The Legacy of Cotton
True Answer	LaLee's Kin: The Legacy of Cotton
Query	Where was the director of film Ronnie Rocket born?
Iter-1 Sentences	s ₁₂ Ronnie Rocket is an unfinished film project written by David Lynch, who also intended to direct it.
Iter-1 Answer	David Lynch
Iter-2 Sentences	s₁ Ronnie Rocket is an unfinished film project written by David Lynch, who also intended to direct it. s₄₉ Title: "Early life of David Lynch". David Lynch was born in Missoula, Montana.
Iter-2 Answer	Missoula, Montana
True Answer	Missoula, Montana

additional round of adjustment. However, increasing the number of iterations can also lead to the accumulation and amplification of errors, resulting in some performance degradation.

6.5. Latency analysis

Fig. 5 shows the latency of each RAG method. We ran all tests on a single H100 GPU and recorded the average time the method took from issuing a query to receiving the final answer. It is worth noting that we used the vLLM [41] engine's API server with 20 concurrent threads, so the times shown in the graph include the waiting time for requests in the queue. In the case of synchronous inference, a typical 2-Iteration AIR-RAG processes a query in approximately 0.8 s. It can be observed that AIR-RAG has slightly higher latency compared to SelfAsk and Iter-RetGen. However, AIR-RAG-1It is significantly faster than both while also delivering better performance.

7. Case study

Table 6 demonstrates retrieval-generation alignment with two examples from HotPotQA. In the first iteration, as both questions require multi-hop reasoning, the retriever fails to retrieve all supporting knowledge using only the questions. In the Oscar nomination example, since LaLee's Kin and Gimme Shelter share a director, the retriever overemphasizes the director information and overlooks the nomination condition, leading to entity confusion. In the Ronnie Rocket example, the retriever fails to move beyond film-level information to the director's biography, remaining at identifying David Lynch without resolving his birthplace, which indicates a missing reasoning hop. In the second iteration, the adapter model reranks and filters the retrieved sentences, guiding the system to the correct answer. More cases are shown in Appendix B.

8. Conclusion

We introduced AIR-RAG, a novel approach to enhancing RAG systems. It refines the retrieval process through iterative feedback, ensuring better alignment between the retriever and the LLM. Unlike previous approaches, AIR-RAG optimizes the entire retrieval pipeline, improving both retrieval ranking and the quality of retrieved documents. We demonstrated that AIR-RAG improves the accuracy and relevance of generated answers by adaptively adjusting the retrieval process based on feedback from the LLM. Through extensive experiments, we showed that our method outperforms existing RAG techniques, providing more efficient solutions.

Limitations

The approach of controlling data flow through sentence selection in the adapter model is one solution under the AIR-RAG framework. Other solutions, such as token-level selection or a hybrid selection across multiple levels like passage, sentence, and token, are also potential research directions, which will be left for future work. Due to resource limitations, our method is mainly validated on open-domain question-answering datasets and fact-checking benchmarks. While briefly tested on the PubHealth dataset, it has not been thoroughly evaluated in some specialized domains such as medicine and law. A key challenge in applying AIR-RAG to these domains is the relative scarcity of domain-specific data compared with publicly available general-domain corpora. Future research could also investigate the applicability of the adapter-based framework in other multi-component systems, such as industrial robotics [42,43].

CRedit authorship contribution statement

Wenhan Han: Writing – review & editing, Writing – original draft, Validation, Methodology, Data curation, Conceptualization. **Xiao Xiao:** Writing – review & editing. **Yaohang Li:** Writing – review & editing. **Jun Wang:** Supervision. **Mykola Pechenizkiy:** Supervision. **Meng Fang:** Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Algorithm details

Algorithms 1–3 provide the detailed procedure of AIR-RAG inference, preference data collection and model training.

Algorithm 1 AIR-RAG Inference Under T-iteration.

Require: Query q , retriever R , adapter M_θ , generator G , iterations T , passages per retrieval k
Ensure: Final answer $\hat{A}$

```

1: function AIR-RAG( $q, R, M_\theta, G, T, k$ )
2:    $q^{(1)} \leftarrow q$ 
3:   for  $t = 1$  to  $T$  do
4:      $\text{Top-}k^{(t)} \leftarrow R(q^{(t)})$ 
5:      $y^{(t)} \leftarrow M_\theta(q^{(t)}, \text{Top-}k^{(t)})$     ▷ Select sentence IDs / sentences
6:     if  $t < T$  then
7:        $q^{(t+1)} \leftarrow q \oplus y^{(t)}$     ▷ Augment query for the next retrieval
8:     end if
9:   end for
10:   $\hat{A} \leftarrow G(q, y^{(T)})$     ▷ Final generation with filtered context
11:  return  $\hat{A}$ 
12: end function

```


Algorithm 2 Offline Preference Collection with Weighted Labels.

Require: Query set Q with gold answers A^* ; retriever R ; generator G ; initial adapter M_{init} ; samples per query n ; passages per retrieval k

Ensure: Weighted preference dataset $D = \{(q, y, w, a)\}$

```

1:  $D \leftarrow \emptyset$ 
2: for all  $q \in Q$  do
3:    $\text{Top-}k \leftarrow R(q)$ 
4:    $\mathcal{Y} \leftarrow \{y_i \sim M_{\text{init}}(q, \text{Top-}k) \mid i = 1..n\}$   $\triangleright$  Sample sentence selections
5:   for  $i = 1$  to  $n$  do  $\triangleright$  First pass: LLM preference
6:      $A_i \leftarrow G(q, y_i)$ 
7:      $\text{is\_correct}_i \leftarrow \text{EVALUATE}(A_i, A_q^*)$ 
8:      $a_i^l \leftarrow \begin{cases} +1, & \text{if } \text{is\_correct}_i \\ -1, & \text{otherwise} \end{cases}$ 
9:   end for
10:  for  $i = 1$  to  $n$  do  $\triangleright$  Second pass: retriever preference via augmented query
11:     $q'_i \leftarrow q \oplus y_i$ 
12:     $\text{Top-}k'_i \leftarrow R(q'_i)$ 
13:     $y'_i \leftarrow M_{\text{init}}(q'_i, \text{Top-}k'_i)$ 
14:     $A'_i \leftarrow G(q, y'_i)$ 
15:     $\text{is\_correct}'_i \leftarrow \text{EVALUATE}(A'_i, A_q^*)$ 
16:     $a_r^i \leftarrow \begin{cases} +1, & \text{if } \text{is\_correct}'_i \\ -1, & \text{otherwise} \end{cases}$ 
17:     $\triangleright$  Fuse preferences into weighted binary label  $(w, a)$ 
18:     $(w, a) \leftarrow \begin{cases} (1.0, +1) & \text{if } (a_i^l = +1, a_r^i = +1) \text{ (Strong Preferred)} \\ (0.5, +1) & \text{if } (a_i^l = +1, a_r^i = -1) \text{ (Weak Preferred)} \\ (0.5, -1) & \text{if } (a_i^l = -1, a_r^i = +1) \text{ (Weak Dispreferred)} \\ (1.0, -1) & \text{if } (a_i^l = -1, a_r^i = -1) \text{ (Strong Dispreferred)} \end{cases}$ 
19:     $D \leftarrow D \cup \{(q, y_i, w, a)\}$ 
20:  end for
21: end for
22: return  $D$ 

```

Algorithm 3 Weighted KTO Update for Adapter M_θ .

Require: Weighted dataset $D = \{(q, y, w, a)\}$; reference policy π_{ref} ; step size η ; batch size m

Ensure: Updated parameters θ

```

1: while not converged do
2:   Sample mini-batch  $B = \{(q_i, y_i, w_i, a_i)\}_{i=1}^m \subset D$ 
3:    $\triangleright$  Biased reference estimate  $\hat{z}_0$  (mismatched pairs)
4:    $\hat{z}_0 \leftarrow \max\left(0, \frac{1}{m} \sum_{i \neq j} \log \frac{\pi_\theta(y_j | q_i)}{\pi_{\text{ref}}(y_j | q_i)}\right)$ 
5:    $\triangleright$  Per-sample utility margin
6:   for  $i = 1$  to  $m$  do
7:      $r_i \leftarrow \log \frac{\pi_\theta(y_i | q_i)}{\pi_{\text{ref}}(y_i | q_i)}$ 
8:      $u_i \leftarrow r_i - \hat{z}_0$ 
9:      $\ell_i \leftarrow w_i \cdot a_i \cdot v(u_i)$   $\triangleright v(\cdot)$  is the KTO value function
10:  end for
11:   $\mathcal{L} \leftarrow -\frac{1}{m} \sum_{i=1}^m \ell_i$   $\triangleright$  Maximize utility  $\rightarrow$  minimize  $-\mathcal{L}$ 
12:   $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$   $\triangleright$  (with LoRA, warmup, etc.)
13: end while
14: return  $\theta$ 

```

Appendix B. Case study

To further analyze how AIR-RAG improves LLM responses compared to other RAG methods, we examined the first 500 samples from the WikiMultiHop, HotpotQA, and PopQA test sets. We selected

samples where AIR-RAG's responses outperformed those of NaiveRAG, Iter-RetGen, and AIR-RAG-1Its simultaneously. Through manual analysis, we attributed AIR-RAG's improvements to the following operations: **Indirect Hint**, **Partial Background Hidden**, **Iterative Retrieval**, **Reranking**, **Retrieval Ablation**, and **Noise Manipulation**. The explanations for these operations are as follows:

- **Indirect Hint** The sentences output by the adapter model to the LLM contain some background information but do not include the answer to the question.
- **Partial Background Hidden** The adapter model deliberately conceals part of the background information to prompt the LLM to produce the desired answer.
- **Iterative Retrieval** AIR-RAG, through iterative retrieval, identifies sentences in the second round that were not retrieved in the first round, and these sentences contain clear answers.
- **Reranking** AIR-RAG selects sentences with essentially the same information in both rounds, both containing clear answers. In the second round, the adapter model slightly adjusts the position of the sentences and a small number of non-critical sentences, enabling the LLM to ultimately provide the correct response.
- **Retrieval Ablation** AIR-RAG conceals all retrieved content and directly prompts the LLM to answer.
- **Noise Manipulation** AIR-RAG selects sentences in both rounds that appear unrelated to the question. However, by adjusting the noisy information, it enables the LLM to provide the correct answer in the second round.

The above attributions represent reasonable hypotheses about the behavior of the adapter model from a human perspective. During sample classification, we found that some samples are difficult to attribute solely to a single operation. For instance, examples categorized as Reranking may also involve Partial Background Hidden and Noise Manipulation. We strive to classify samples based on the most proximate and apparent reasons. These analyses are intended to provide a more detailed understanding of AIR-RAG's mechanisms, rather than to claim that AIR-RAG achieves or is limited to these specific functionalities. Table B.7 presents the distribution.

B.1. Why are AIR-RAG's iterative retrieval and reranking different from other RAG methods based on these operations?

Table B.8 illustrates an example of the reranking (iterative retrieval involved) operation of AIR-RAG. In the first round, although the retriever identified the answer and the adapter model passed the key sentence to the LLM, the LLM still produced an incorrect response. Similarly, NaiveRAG and Iter-RetGen also failed to provide the correct answer. In the second round, *Gimme Shelter* related content was ranked lower in the retrieval results, prompting the adapter model to remove related sentences from the LLM's input, ultimately guiding the LLM to correctly answer the question.

This clearly demonstrates how AIR-RAG's Iterative Retrieval and Reranking differ from traditional methods. In conventional approaches,

Table B.7

Attribution analysis of the improvement brought by AIR-RAG. The numbers represent the quantity of test samples (out of 500) across various test sets where AIR-RAG's answers outperform those of NaiveRAG, Iter-RetGen, and AIR-RAG-1Its simultaneously.

Attribution	WikiMultiHop	HotpotQA	PopQA
Indirect Hint	4	3	1
Partial Background Hidden	4	–	–
Iterative Retrieval	4	7	1
Reranking	1	3	2
Retrieval Ablation	2	1	2
Noise Manipulation	2	–	–

Table B.8

Case 1: Improvement from Reranking by AIR-RAG.

Query	Which film was Oscar nominated, LaLee's Kin: The Legacy of Cotton or Gimme Shelter , the 1970 Rolling Stones documentary?
Iter-1 Sentences	s_1 LaLee's Kin: The Legacy of Cotton is a 2001 American documentary film directed by Deborah Dickson, Susan Frömke, and Albert Maysles s_2 It was nominated for Best Documentary Feature at the 74th Academy Awards s_3 LaLee's Kin: The Legacy of Cotton has two storylines, both of which show the impoverished life of residents in the American South s_4 The documentary draws the connection—a vicious cycle—between poverty and the lack of education opportunities for Black people living in the Mississippi Delta, over 150 years after the abolition of slavery s_8 LaLee's Kin: The Legacy of Cotton garnered a nomination for Best Documentary Feature at both the 74th Academy Awards and the 2002 Independent Spirit Award s_17 LaLee's Kin: The Legacy of Cotton received highly positive reviews by critics s_24 Gimme Shelter is a 1970 British-American documentary film directed by Albert and David Maysles and Charlotte Zwerin, chronicling the last weeks of The Rolling Stones' 1969 US tour, which culminated in the disastrous Altamont Free Concert s_26 Gimme Shelter was screened out of competition as the opening film of the 1971 Cannes Film Festival
Iter-1 Answer	Gimme Shelter
Iter-2 Sentences	s_1 LaLee's Kin: The Legacy of Cotton is a 2001 American documentary film directed by Deborah Dickson, Susan Frömke, and Albert Maysles s_2 It was nominated for Best Documentary Feature at the 74th Academy Awards s_3 LaLee's Kin: The Legacy of Cotton has two storylines, both of which show the impoverished life of residents in the American South s_4 The documentary draws the connection—a vicious cycle—between poverty and the lack of education opportunities for Black people living in the Mississippi Delta, over 150 years after the abolition of slavery s_8 LaLee's Kin: The Legacy of Cotton garnered a nomination for Best Documentary Feature at both the 74th Academy Awards and the 2002 Independent Spirit Award s_17 LaLee's Kin: The Legacy of Cotton received highly positive reviews by critics s_18 The documentary received a score of 78 out of 100 on Metacritic based on 5 reviews s_19 The New York Times critic A. O. Scott praised the film as an exemplary work of cinéma vérité that allows its subjects to speak for themselves
Iter-2 Answer	LaLee's Kin: The Legacy of Cotton
NaiveRAG Answer	Gimme Shelter
Iter-RetGen Answer	Gimme Shelter
Direct Answer	LaLee's Kin: The Legacy of Cotton
True Answer	LaLee's Kin: The Legacy of Cotton

retrieval and reranking typically prioritize all relevant information based on human preferences. In contrast, AIR-RAG adjusts the presented content according to the LLM's comprehension capabilities and processing characteristics.

B.2. Other cases

Tables B.10–B.12 present additional examples, including some operations by the adapter model that are relatively difficult to understand from a human perspective. Many of the adapter model's operations are conducted in ways that are not readily comprehensible to humans. These operations stem from the adapter model's understanding of the characteristics of the LLM and retriever, which it gained through preference learning. This cannot simply be attributed to iterative retrieval or reranking, as the results of AIR-RAG are clearly superior to those of Iter-RetGen. Even in cases where the retriever successfully retrieved the correct answer, the LLM sometimes failed to produce the correct response due

to its inherent limitations. The adapter model, by understanding some of these underlying characteristics of the LLM, was able to guide it toward generating the correct answer.

B.3. How to understand AIR-RAG's learning of retriever preferences?

The learning of retriever preference by the adapter model is also reflected in the performance differences between AIR-RAG's Iterative Retrieval and other Iterative RAG methods. Table B.9 presents a typical example of Iterative Retrieval. Although Iter-RetGen also employs query augmentation for multiple iterations to enhance retrieval quality, it still fails to provide the LLM with an accurate answer. In contrast, AIR-RAG implicitly learns the retriever preference, understanding the characteristics of the retriever. This allows it to select specific information to incorporate into the query, thereby promoting information containing the precise answer to the top of the retrieval results in the second round of retrieval.

Table B.9

Case 2: Improvement from Iterative Retrieval by AIR-RAG.

Query	The author of Sexual Politics attended which British University?
Iter-1 Sentences	s_8 Kate Millett Katherine Murray Millett (September 14, 1934 – September 6, 2017) was an American feminist writer, educator, artist, and activist. s_10 She has been described as “a seminal influence on second-wave feminism” and is best known for her book Sexual Politics (1970), which was based on her doctoral dissertation at Columbia University. s_27 Sexual Politics originated as Millett’s PhD dissertation and was published in 1970, the same year that she was awarded her doctorate from Columbia University.
Iter-1 Answer	Columbia University
Iter-2 Sentences	s_1 Kate Millett Katherine Murray Millett (September 14, 1934 – September 6, 2017) was an American feminist writer, educator, artist, and activist. s_2 She attended Oxford University and was the first American woman to be awarded a degree with first-class honors after studying at St Hilda’s College, Oxford. s_3 She has been described as “a seminal influence on second-wave feminism” and is best known for her book Sexual Politics (1970), which was based on her doctoral dissertation at Columbia University.
Iter-2 Answer	St Hilda’s College, Oxford
NaiveRAG Answer	Cambridge University
Iter-RetGen Answer	Cambridge
Direct Answer	The author of Sexual Politics attended the University of Oxford .
True Answer	Oxford

Table B.10

An example of a Partial Background Hidden case. In this case, it can be observed that although in the first round AIR-RAG’s adapter model accurately provided the LLM with evidence that could have easily led to the correct answer, the LLM still answered incorrectly. Similarly, NaiveRAG, Iter-RetGen, and Direct all failed to provide the correct answer. However, in the second round, a reversal occurred: the adapter model removed all information related to The Ballad of Josie and successfully guided the LLM to answer the question correctly.

Query	Which film was released earlier, Moment Of Danger or The Ballad Of Josie?
Iter-1 Sentences	s_17 Moment of Danger (also known as Malaga) is a 1960 crime drama film starring Trevor Howard, Dorothy Dandridge, and Edmund Purdom. s_1 The Ballad of Josie is a 1967 Technicolor American comedy western film directed by Andrew V. McLaglen and starring Doris Day, Peter Graves, and George Kennedy. s_10 The Ballad of Josie is a 1967 Technicolor American comedy western film directed by Andrew V. McLaglen and starring Doris Day, Peter Graves, and George Kennedy.
Iter-1 Answer	The Ballad of Josie
Iter-2 Sentences	s_1 Moment of Danger (also known as Malaga) is a 1960 crime drama film starring Trevor Howard, Dorothy Dandridge, and Edmund Purdom. s_4 The film proved to be the final completed film for Dorothy Dandridge. s_5 Starting with a wordless jewel heist pulled off by thief Peter Curran and locksmith John Bain, Curran then double-crosses his accomplice, dumps his lover Gianna, and escapes with his ill-gotten gains. s_10 It was filmed in Europe in the late months of 1959. s_12 The film proved to be the final completed film for Dorothy Dandridge.
Iter-2 Answer	Moment of Danger
NaiveRAG Answer	The Ballad of Josie was released earlier than Moment of Danger.
Iter-RetGen Answer	The Ballad of Josie
Direct Answer	The Ballad Of Josie was released earlier.
True Answer	Moment Of Danger

Table B.11

An example of an Indirect Hint case. In this case, the adapter model provided the name of Ronnie Rocket's director, David Lynch, in the first round of selected sentences. However, the LLM was clearly unable to provide a correct answer. In the second round, the adapter model still did not provide David Lynch's specific birthplace but adjusted some cues: "Title: 'Early life of David Lynch'." This activated the LLM's parameter knowledge, leading to the correct answer.

Query	Where was the director of film Ronnie Rocket born?
Iter-1 Sentences	s_0 Title: "Ronnie Rocket". s_12 Ronnie Rocket is an unfinished film project written by David Lynch, who also intended to direct it. s_13 Begun after the success of Lynch's 1977 film "Eraserhead", "Ronnie Rocket" was shelved after Lynch felt he would be unable to find financial backing for the project. s_34 After releasing 1977's "Eraserhead", a black-and-white surrealist film and his début feature-length production, Lynch began work on the screenplay for "Ronnie Rocket". s_30 The project has also suffered setbacks due to the bankruptcy of several potential backers; both Dino De Laurentiis's De Laurentiis Entertainment Group and Francis Ford Coppola's American Zoetrope were attached to the project at different times; both production companies went bankrupt before work could begin.
Iter-1 Answer	David Lynch
Iter-2 Sentences	s_1 Ronnie Rocket is an unfinished film project written by David Lynch, who also intended to direct it. s_49 Title: "Early life of David Lynch".
Iter-2 Answer	David Lynch was born in Missoula, Montana .
NaiveRAG Answer	Ronnie Rocket was born in Bismarck, North Dakota, United States .
Iter-RetGen Answer	Bismarck, North Dakota
Direct Answer	The director of film Ronnie Rocket was born in New York City, New York, United States .
True Answer	Missoula, Montana

Table B.12

An example of a Noise Manipulation case. In this case, the adapter model did not provide any sentences related to the two films. In the first round, AIR-RAG, like other methods, also answered incorrectly. However, in the second round, the adapter model's manipulation of the noise altered the LLM's response, leading to the correct answer.

Query	Which film has the director who was born earlier, Retalhos Da Vida De Um Médico or The Shooting?
Iter-1 Sentences	s_1 "Fogo na Noite Escura" (1943), at the collection "Novos Prosadores" (1943), by "Coimbra Editora". s_2 Besides over 30 titles, along his fifty years of intensive literary life, not only wrote "neo-realistic" novels, as "Casa da Malta" (1945), "Minas de S. Francisco" (1946), Retalhos da Vida de um Médico (1949 and 1963), "A Noite e a Madrugada" (1950), "O Trigo e o Joio" (1954), but also "urban themes," contemporary fiction, as in "O Homem Disfarçado" (1957), "Cidade Solitária" (1959), "Domingo à Tarde" (1961, José Lins do Rego Prize), "Os Clandestinos" (1972), "Resposta a Matilde" (1980) or "O Rio Triste" (1982, Fernando Chinaglia). s_3 Title: "Autran Dourado". s_4 In the Portuguese "sprachraum". s_5 In 2001, Brazilian filmmaker Suzana Amaral released the film "Uma Vida em Segredo". s_6 It was based on the novel of same title by Autran Dourado.
Iter-1 Answer	The Shooting
Iter-2 Sentences	s_2 Besides over 30 titles, along his fifty years of intensive literary life, not only wrote "neo-realistic" novels, as "Casa da Malta" (1945), "Minas de S. Francisco" (1946), Retalhos da Vida de um Médico (1949 and 1963), "A Noite e a Madrugada" (1950), "O Trigo e o Joio" (1954), but also "urban themes," contemporary fiction, as in "O Homem Disfarçado" (1957), "Cidade Solitária" (1959), "Domingo à Tarde" (1961, José Lins do Rego Prize), "Os Clandestinos" (1972), "Resposta a Matilde" (1980) or "O Rio Triste" (1982, Fernando Chinaglia). s_1 "Fogo na Noite Escura" (1943), at the collection "Novos Prosadores" (1943), by "Coimbra Editora".
Iter-2 Answer	Retalhos Da Vida De Um Médico
NaiveRAG Answer	The director of The Shooting was born earlier than the director of Retalhos Da Vida De Um Médico .
Iter-RetGen Answer	The Shooting
Direct Answer	The Shooting
True Answer	Retalhos Da Vida De Um Médico

Appendix C. Data domain distribution

Fig. C.6 shows the visualization of the semantic vector distributions of the training and test sets used in our study. We use the Qwen3-Embedding-8B model to compute the semantic embeddings of all queries. As can be seen, the training sets of TriviaQA and HotpotQA are clearly distinguishable from the other four test sets in terms of their semantic distributions. Therefore, using these four test sets to evaluate the out-of-domain performance of AIR-RAG carries a low risk of data leakage.

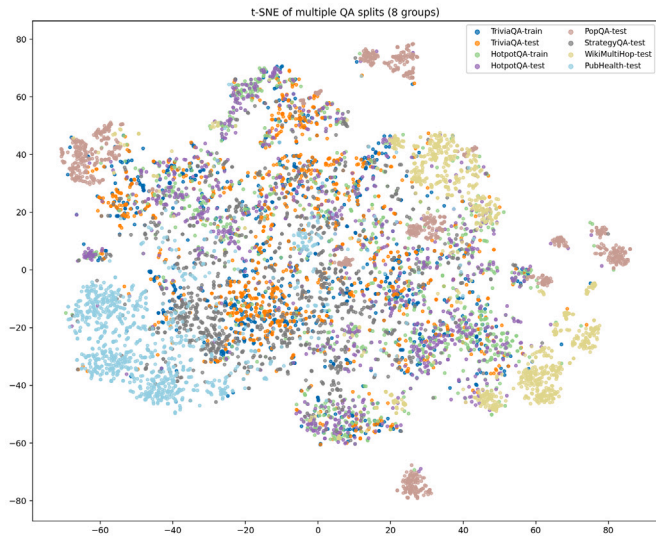


Fig. C.6. Visualization of domain clusters of training and test sets.

Data availability

Data will be made available on request.

References

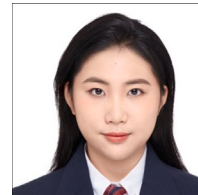
- [1] M. Yasunaga, A. Aghajanyan, W. Shi, R. James, J. Leskovec, P. Liang, M. Lewis, L. Zettlemoyer, W.-T. Yih, Retrieval-augmented multimodal language modeling. *arXiv:2211.12561*, Jun 2023.
- [2] Y. Zhu, P. Zhang, C. Zhang, Y. Chen, B. Xie, Z. Liu, J.-R. Wen, Z. Dou, INTERS: unlocking the power of large language models in search with instruction tuning, in: L.-W. Ku, A. Martins, V. Srikumar (Eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Bangkok, Thailand, Association for Computational Linguistics, 2024, pp. 2782–2809.
- [3] G. Xiong, Q. Jin, Z. Lu, A. Zhang, Benchmarking retrieval-augmented generation for medicine. *arXiv:2402.13178*, Feb 2024.
- [4] R. Xu, W. Shi, Y. Yu, Y. Zhuang, B. Jin, M.D. Wang, J. Ho, C. Yang, RAM-EHR: retrieval augmentation meets clinical predictions on electronic health records, in: L.-W. Ku, A. Martins, V. Srikumar (Eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, Bangkok, Thailand, Association for Computational Linguistics, 2024, pp. 754–765.
- [5] Z. Yue, H. Zeng, L. Shang, Y. Liu, Y. Zhang, D. Wang, Retrieval augmented fact verification by synthesizing contrastive arguments, in: L.-W. Ku, A. Martins, V. Srikumar (Eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Bangkok, Thailand, Association for Computational Linguistics, 2024, pp. 10331–10343.
- [6] A. Asai, Z. Wu, Y. Wang, A. Sil, H. Hajishirzi, Self-RAG: learning to retrieve, generate, and critique through self-reflection, in: *The Twelfth International Conference on Learning Representations*, 2023.
- [7] Z. Ke, W. Kong, C. Li, M. Zhang, Q. Mei, M. Bendersky, Bridging the preference gap between retrievers and LLMs, in: L.-W. Ku, A. Martins, V. Srikumar (Eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Bangkok, Thailand, Association for Computational Linguistics, 2024, pp. 10438–10451.
- [8] O. Press, M. Zhang, S. Min, L. Schmidt, N. Smith, M. Lewis, Measuring and narrowing the compositionality gap in language models, in: H. Bouamor, J. Pino, K. Bali (Eds.), *Findings of the Association for Computational Linguistics: EMNLP 2023*, Singapore, Association for Computational Linguistics, 2023, pp. 5687–5711, <https://doi.org/10.18653/v1/2023.findings-emnlp.378>
- [9] Z. Jiang, F. Xu, L. Gao, Z. Sun, Q. Liu, J. Dwivedi-Yu, Y. Yang, J. Callan, G. Neubig, Active retrieval augmented generation, in: H. Bouamor, J. Pino, K. Bali (Eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Singapore, Association for Computational Linguistics, 2023, pp. 7969–7992, <https://doi.org/10.18653/v1/2023.emnlp-main.495>
- [10] Z. Shao, Y. Gong, Y. Shen, M. Huang, N. Duan, W. Chen, Enhancing retrieval-augmented large language models with iterative retrieval-generation synergy, in: H. Bouamor, J. Pino, K. Bali (Eds.), *Findings of the Association for Computational Linguistics: EMNLP 2023*, Singapore, Association for Computational Linguistics, 2023, pp. 9248–9274, <https://doi.org/10.18653/v1/2023.findings-emnlp.620>
- [11] J. Feng, C. Tao, X. Geng, T. Shen, C. Xu, G. Long, D. Zhao, D. Jiang, Synergistic interplay between search and large language models for information retrieval, in: L.-W. Ku, A. Martins, V. Srikumar (Eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Bangkok, Thailand, Association for Computational Linguistics, 2024, pp. 9571–9583.
- [12] Z. Shi, S. Zhang, W. Sun, S. Gao, P. Ren, Z. Chen, Z. Ren, Generate-then-ground in retrieval-augmented generation for multi-hop question answering, in: L.-W. Ku, A. Martins, V. Srikumar (Eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Bangkok, Thailand, Association for Computational Linguistics, 2024, pp. 7339–7353.
- [13] X. Ma, Y. Gong, P. He, H. Zhao, N. Duan, Query rewriting for retrieval-augmented large language models.
- [14] Z. Wu, Y. Luan, H. Rashkin, D. Reitter, H. Hajishirzi, M. Ostendorf, G.S. Tomar, CONQR: conversational query rewriting for retrieval with reinforcement learning. *arXiv:2112.08558*, Oct 2022.
- [15] K. Mao, Z. Dou, F. Mo, J. Hou, H. Chen, H. Qian, Large language models know your contextual search intent: a prompting framework for conversational search. *arXiv:2303.06573*, Oct 2023.
- [16] L. Zhang, Y. Yu, K. Wang, C. Zhang, ARL2: aligning retrievers with black-box large language models via self-guided relevance labeling, in: L.-W. Ku, A. Martins, V. Srikumar (Eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Bangkok, Thailand, Association for Computational Linguistics, 2024, pp. 3708–3719.
- [17] J. Yoon, Y. Chen, S. Arik, T. Pfister, Search-adaptor: embedding customization for information retrieval, in: L.-W. Ku, A. Martins, V. Srikumar (Eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Bangkok, Thailand, Association for Computational Linguistics, 2024, pp. 12230–12247.
- [18] Z. Liao, H. Yu, J. Li, J. Wang, W. Zhang, D2llm: decomposed and distilled large language models for semantic search, in: L.-W. Ku, A. Martins, V. Srikumar (Eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Bangkok, Thailand, Association for Computational Linguistics, 2024, pp. 14798–14814.
- [19] K. Luo, Z. Liu, S. Xiao, T. Zhou, Y. Chen, J. Zhao, K. Liu, Landmark embedding: a chunking-free embedding method for retrieval augmented long-context large language models, in: L.-W. Ku, A. Martins, V. Srikumar (Eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Bangkok, Thailand, Association for Computational Linguistics, 2024, pp. 3268–3281.
- [20] P. Zhang, Z. Liu, S. Xiao, Z. Dou, J.-Y. Nie, A multi-task embedder for retrieval augmented llms, in: L.-W. Ku, A. Martins, V. Srikumar (Eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Bangkok, Thailand, Association for Computational Linguistics, 2024, pp. 3537–3553.
- [21] Z. Zhang, M. Fang, L. Chen, Retrievalqa: assessing adaptive retrieval-augmented generation for short-form open-domain question answering, in: L.-W. Ku, A. Martins, V. Srikumar (Eds.), *Findings of the Association for Computational Linguistics: ACL 2024*, Bangkok, Thailand, Association for Computational Linguistics, 2024, pp. 6963–6975, <https://doi.org/10.18653/v1/2024.findings-acl.415>, <https://aclanthology.org/2024.findings-acl.415/>.
- [22] H. Qian, Z. Liu, K. Mao, Y. Zhou, Z. Dou, Grounding language model with chunking-free in-context retrieval, in: L.-W. Ku, A. Martins, V. Srikumar (Eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Bangkok, Thailand, Association for Computational Linguistics, 2024, pp. 1298–1311.
- [23] S. Xu, L. Pang, M. Yu, F. Meng, H. Shen, X. Cheng, J. Zhou, Unsupervised information refinement training of large language models for retrieval-augmented generation. *arXiv:2402.18150*, Jun 2024.
- [24] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, G. Lample, LLaMA: open and efficient foundation language models. *arXiv:2302.13971*, Feb 2023.
- [25] K. Zhu, X. Feng, X. Du, Y. Gu, W. Yu, H. Wang, Q. Chen, Z. Chu, J. Chen, B. Qin, An information bottleneck perspective for effective noise filtering on retrieval-augmented generation, in: L.-W. Ku, A. Martins, V. Srikumar (Eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Bangkok, Thailand, Association for Computational Linguistics, 2024, pp. 1044–1069.
- [26] F. Fang, Y. Bai, S. Ni, M. Yang, X. Chen, R. Xu, Enhancing noise robustness of retrieval-augmented language models with adaptive adversarial training, in: L.-W. Ku, A. Martins, V. Srikumar (Eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Bangkok, Thailand, Association for Computational Linguistics, 2024, pp. 10028–10039.

- [27] K. Ethayarajh, W. Xu, N. Muennighoff, D. Jurafsky, D. Kiela, KTO: model alignment as prospect theoretic optimization. [arXiv:2402.01306](https://arxiv.org/abs/2402.01306), Sep 2024.
- [28] P. Christiano, J. Leike, T.B. Brown, M. Martic, S. Legg, D. Amodei, Deep reinforcement learning from human preferences. [arXiv:1706.03741](https://arxiv.org/abs/1706.03741), 2023.
- [29] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C.D. Manning, C. Finn, Direct preference optimization: your language model is secretly a reward model. [arXiv:2305.18290](https://arxiv.org/abs/2305.18290), 2024.
- [30] M. Joshi, E. Choi, D. Weld, L. Zettlemoyer, Triviaqa: a large scale distantly supervised challenge dataset for reading comprehension, in: R. Barzilay, M.-Y. Kan (Eds.), Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Vancouver, Canada, Association for Computational Linguistics, 2017, pp. 1601–1611, <https://doi.org/10.18653/v1/P17-1147>.
- [31] A. Mallen, A. Asai, V. Zhong, R. Das, D. Khashabi, H. Hajishirzi, When Not to trust language models: investigating effectiveness of parametric and non-parametric memories. [arXiv:2212.10511](https://arxiv.org/abs/2212.10511), Jul 2023.
- [32] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W.W. Cohen, R. Salakhutdinov, C.D. Manning, HotpotQA: a dataset for diverse, explainable multi-hop question answering. [arXiv:1809.09600](https://arxiv.org/abs/1809.09600), Sep 2018.
- [33] X. Ho, A.-K. Duong Nguyen, S. Sugawara, A. Aizawa, Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps, in: D. Scott, N. Bel, C. Zong (Eds.), Proceedings of the 28th International Conference on Computational Linguistics, Barcelona, Spain (Online), International Committee on Computational Linguistics, 2020, pp. 6609–6625, <https://doi.org/10.18653/v1/2020.coling-main.580>.
- [34] N. Kotonya, F. Toni, Explainable Automated fact-checking for public health claims. [arXiv:2010.09926](https://arxiv.org/abs/2010.09926), Oct 2020.
- [35] M. Geva, D. Khashabi, E. Segal, T. Khot, D. Roth, J. Berant, Did Aristotle use a laptop? a question answering benchmark with implicit reasoning strategies, *Trans. Assoc. Comput. Linguistics* 9 (2021) 346–361, https://doi.org/10.1162/TACL_A_00370.
- [36] X. Zhang, Y. Song, Y. Wang, S. Tang, X. Li, Z. Zeng, Z. Wu, W. Ye, W. Xu, Y. Zhang, X. Dai, S. Zhang, Q. Wen, RAGLAB: a modular and research-oriented unified framework for retrieval-augmented generation. [arXiv:2408.11381](https://arxiv.org/abs/2408.11381), Aug 2024.
- [37] S. Xu, L. Pang, H. Shen, X. Cheng, T.-S. Chua, Search-in-the-chain: interactively enhancing large language models with search for knowledge-intensive tasks. [arXiv:2304.14732](https://arxiv.org/abs/2304.14732), Feb 2024.
- [38] K. Santhanam, O. Khattab, J. Saad-Falcon, C. Potts, M. Zaharia, Colbertv2: effective and efficient retrieval via lightweight late interaction, in: M. Carpuat, M.-C. de Marneffe, I.V. Meza Ruiz (Eds.), Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Seattle, United States, Association for Computational Linguistics, 2022, pp. 3715–3734, <https://doi.org/10.18653/v1/2022.naacl-main.272>.
- [39] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, W.-T. Yih, Dense passage retrieval for open-domain question answering, in: B. Webber, T. Cohn, Y. He, Y. Liu (Eds.), Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), Online, Association for Computational Linguistics, 2020, pp. 6769–6781, <https://doi.org/10.18653/v1/2020.emnlp-main.550>.
- [40] H. Tan, F. Sun, W. Yang, Y. Wang, Q. Cao, X. Cheng, Blinded by generated contexts: how language models merge generated and retrieved contexts when knowledge conflicts? in: L.-W. Ku, A. Martins, V. Srikumar (Eds.), Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Bangkok, Thailand, Association for Computational Linguistics, 2024, pp. 6207–6227.
- [41] Y. Pisarchyk, J. Lee, Efficient memory management for deep neural net inference, [arXiv:2001.03288v3](https://arxiv.org/abs/2001.03288v3), Jan 2020.
- [42] H. Bilal, M.S. Obaidat, M.S. Aslam, J. Zhang, B. Yin, K. Mahmood, Online fault diagnosis of industrial robot using iort and hybrid deep learning techniques: an experimental approach, *IEEE Internet Things J.* 11 (19) (2024) 31422–31437, <https://doi.org/10.1109/JIOT.2024.3418352>.
- [43] H. Bilal, A. Rehman, M.S. Aslam, I. Ullah, W.-J. Chang, N. Kumar, A.M. Almuhaideb, Hybrid trafficai: a generative AI framework for real-time traffic simulation and adaptive behavior modeling, *IEEE Trans. Intell. Transp. Syst.* PP (2025) 1–17.

Author biography



Wenhan Han obtained his Bachelor's degree in Computer Science and Technology from South China University of Technology in 2018. He received his Master of Science degree from Eindhoven University of Technology in 2022 and is currently pursuing a Ph.D. at Eindhoven University of Technology under the supervision of Prof. Mykola Pechenizkiy. His research interests include multilingual large language models, retrieval-augmented generation, and reinforcement learning.



Xiao Xiao was born in Yichang, China, in 2000, and received Master of Science degree in Disability Design and Innovation from University College London, UK, in 2023. She is currently a Ph.D. student in the Department of Computer Science at the University of Liverpool, UK. Her current research interests include cross-lingual consistency and bias in medical applications of large language models.



Yaohang Li received the dual bachelor's degrees in computer science from Northeastern University, Shenyang, China, and in software engineering (Honours) from the University of Technology Sydney, Sydney, Australia, in 2025. He is currently a Software Developer on the AI team at EML, Sydney, Australia. His research interests include large language models, model optimization, and the efficient deployment of AI systems.



Jun Wang is Chair Professor, Computer Science, University College London, and Founding Director of MSc Web Science and Big Data Analytics. He is also Co-founder and Chief Scientist in MediaGamma Ltd, a UCL start-up company focusing on AI for intelligent audience decision making. Prof. Jun Wang's main research interests are in the areas of AI and intelligent systems, including (multiagent) reinforcement learning, deep generative models, and their diverse applications on information retrieval, recommender systems and personalization, data mining, smart cities, bot planning, computational advertising etc.



Mykola Pechenizkiy is a Full Professor at the department of Mathematics and Computer Science, Eindhoven University of Technology (TU/e), where he holds the Data Mining Chair. His research interests include data science, knowledge discovery and data mining, responsible analytics, including ethics/discrimination-awareness, context-aware predictive analytics, handling concept drift and reoccurring contexts, automation of feature construction and analytics on evolving networks.



Dr. Meng Fang currently works at the University of Liverpool, focusing on developing intelligent agents capable of human-like learning, reasoning, and decision-making. His research areas include natural language processing (NLP) and reinforcement learning/machine learning (RL/ML). He has published more than 60 papers in leading AI and NLP venues such as NeurIPS, ACL, and ICLR, and has received several distinctions, including the Best Paper Award at the Learning on Graphs (LoG 2022) conference.